Yash's CP Cheatsheet

C++17 · COMBINED TEMPLATE · QUICK REFERENCE

SETUP Trigger & Compilation

```
// Type prefix in VS Code → Tab to expand Prefix: hello // Local – enables debug() output via algo/debug.h g++ -DLOCAL -std=c++17 -O2 -Wall -o sol sol.cpp // Judge – debug() becomes no-op (42), zero overhead g++ -std=c++17 -O2 -o sol sol.cpp
```

TYPES Type Aliases — BOTH

`ll`	**long long int** — use for everything by default
`ull`	**unsigned long long** — when values can't be negative
`ld`	**long double** — geometry / precision floats
`vi / vvi`	vector<ll> / 2D vector<ll> — **ll-based**
`vint`	vector<int> — explicit int when needed
`ii / pii`	pair<ll,ll> / pair<int,int>
`vp`	vector<ii> — list of ll pairs

Convention: `vi` = `vector<ll>` here. Use `vint` when you explicitly need ints (e.g. graph adjacency lists).

MACROS Shorthand Macros — BOTH

`pb`	**push_back**
`F / S`	**first / second** on any pair
`all(v)`	(v).begin(), (v).end() — for STL algorithms
`srt(v)`	sort **ascending**
`rsrt(v)`	sort **descending** — rbegin/rend
`endl`	**'\n'** — never use std::endl (flushes = slow)

LOOPS rep() & per() — SNIPPET 1 + ADDED

`rep(i, a, b)`	i = a … b−1 **forward**, i is ll
`per(i, a, b)`	i = b−1 … a **reverse**, i is ll

```
// Standard 0-indexed rep(i, 0, n) cout << a[i] << ' '; // Reverse (e.g. suffix DP, stack pop order) per(i, 0, n) process(a[i]); // 2D grid rep(i, 0, n) rep(j, 0, m) cin >> g[i][j]; // Non-zero start rep(i, 1, n+1) prefix[i] = prefix[i-1] + a[i-1];
```

I/O vin() / pin() / vin2() — Safe Helpers — SNIPPET 2 + ADDED

`vin(a, n)`	Resize vi to n then read — **never forgets resize**
`pin(a)`	Print vi, space-sep, **no trailing space**, newline
`vin2(g, n, m)`	Read full 2D grid — **safe assign + nested read**

```
vi a; vin(a, n); // safe 1-liner pin(a); // "1 2 3\n" no trailing space vvi g; vin2(g, n, m); // full grid input
```

Why not `cin >> v` directly? It silently reads garbage if you forget to resize first. `vin()` always resizes, so it's impossible to get that bug.

I/O Generic cin / cout Operators — SNIPPET 1

`cin >> pair`	Reads p.first then p.second automatically
`cout << pair`	Prints "a b"
`cout << vector`	Space-sep, **no trailing space**, auto newline

```
// Pairs ii p; cin >> p; cout << p; // "x y" // Vector of pairs vector<ii> v(n); cin >> v; cout << v; // Pre-size required! Use vin() to be safe vi a(n); cin >> a; // OK – already sized
```

UTILS chmin / chmax — ADDED

`chmin(a, b)`	a = min(a,b) — returns **true** if a changed
`chmax(a, b)`	a = max(a,b) — returns **true** if a changed

```
// DP transition – replaces verbose if-block chmin(dp[i], dp[j] + cost); // Running answer ll ans = 0; rep(i, 0, n) chmax(ans, a[i]); // Trigger rebuild only on improvement if (chmin(best, candidate)) rebuild();
```

CONSTANTS Built-in Constants — SNIPPET 1

`INF`	**1e18** as ll — safe ll infinity
`INF32`	**1e9** as int — safe int infinity
`MOD`	**1e9 + 7** — standard modulus

⚠ **Overflow:** `INF + INF` overflows if both are int. Always use ll `INF` in ll contexts. When adding to INF, cast first: `(ll)INF32 + x`.

DEBUG debug() Macro — SNIPPET 2

`debug(x)`	Prints variable name + value (only with **-DLOCAL**)
`debug(x, y, z)`	Variadic — any number of args
`On judge`	Compiles to **42** — optimized away, zero cost

```
ll x = 5; vi v = {1,2,3}; debug(x, v); // local: [debug] x=5, v=[1,2,3] // judge: 42; ← removed by optimizer
```

PATTERNS Everyday CP Patterns — Using This Template

Read n longs (safe)	`vi a; vin(a, n);`
Read n pairs	`vp v(n); cin >> v;` — generic operator handles it
Read 2D grid (safe)	`vvi g; vin2(g, n, m);`
Sort pairs by second	`sort(all(v), [](ii a, ii b){ return a.S < b.S; });`
Prefix sum	`vi pre(n+1, 0); rep(i,0,n) pre[i+1] = pre[i] + a[i];`
Frequency map	`map<ll,ll> freq; for (auto x : a) freq[x]++;`
Binary search on answer	`ll lo=0, hi=1e9; while(lo<hi){ ll mid=(lo+hi)/2; check(mid)?hi=mid:lo=mid+1; }`
Modular add (safe)	`ans = (ans + x % MOD + MOD) % MOD;` — +MOD handles negatives
Running max / min	`ll ans=0; rep(i,0,n) chmax(ans, a[i]);`
Reverse traversal	`per(i, 0, n) dp[i] = dp[i+1] + a[i];`
Sort descending	`rsrt(a);`
Print vector clean	`pin(a);` — no trailing space, judge-safe